Entropic Fusion

Why It Exists, Why It Is Fused, and the EFI System Built on Top

Architecture definition paper
EFI 1.0.0-beta.5 / EFI_ITER_021
Research prototype

Abstract

Entropic Fusion was developed around a practical long-term confidentiality problem: **Harvest Now, Decrypt Later (HNDL)**. An attacker does not need to decrypt sensitive material today. It may be enough to collect encrypted files now, keep them, and wait for future quantum computers or other cryptanalytic advances that could break the public-key mechanism that protected their data keys.

EFI therefore uses NIST-standardised post-quantum ML-KEM for key establishment now, while retaining AES-256-GCM as the established conventional authenticated cipher for the actual file data. The defining research construction, however, is not merely "conventional data encryption plus post-quantum key establishment". It is a deliberately coupled root:

> **Entropic Notation + ML-KEM-768 + ML-KEM-1024 = Entropic Fusion**

The Entropic relation and the two root KEM sessions are not arranged as detachable outer and inner encryption layers. In the legitimate private recovery path, the complete Entropic witness must verify before the fusion secret is derived. That fusion secret releases both wrapped root ML-KEM private keys. The two KEM ciphertexts can then be decapsulated, and the final root combiner binds both shared secrets, the EF-gated session contribution, the complete ciphertext transcript and application context into one 256-bit root key.

The design objective is therefore **joint dependency rather than sequential peeling**. The architecture is intended to make the Entropic relation and dual post-quantum KEM sessions part of the same root-recovery problem, rather than independent stages that can simply be solved one after another. The current B.23.1 implementation demonstrates that dependency on the official private path, but this paper does not overstate it as a theorem that every possible attacker must independently break all three components simultaneously. A documented raw-KEM-secret oracle limitation remains the main target of the next root research branch.

Entropic Fusion Integrated, or EFI, is the wider application built on that root:

Entropic Fusion
+ Entropic Tetration: Standard / Tetration / Pentation
+ Entropic Chain / EC100
+ AES-256-GCM
+ metadata-private streaming and graduated padding
+ optional EFIPRIV4 deniable storage and deliberate duress
= EFI

ML-KEM supplies the NIST-standardised post-quantum key-establishment component. AES-256-GCM supplies conventional authenticated bulk-data encryption. The Entropic

Notation, Entropic Fusion, Tetration/Pentation and Entropic Chain constructions remain experimental research objects pending independent cryptanalysis and formal analysis.

1. Why EFI was developed, and what Entropic Fusion means

HNDL in plain language

HNDL means Harvest Now, Decrypt Later. An adversary copies encrypted material today and stores it, betting that future quantum computers or other advances will make the public-key protection of its keys breakable later.

The important point is timing. The attack can begin years before the attacker has the ability to decrypt anything. If the data must remain confidential for ten, twenty or thirty years, it matters whether the key-establishment mechanism used today is expected to survive the computing capabilities of tomorrow.

EFI addresses that threat by using post-quantum ML-KEM for key establishment. It does not rely on classical RSA or elliptic-curve key exchange for the root. The actual file content is still protected with AES-256-GCM, because symmetric bulk encryption and public-key key establishment have different jobs.

The defensible claim is therefore:

> **EFI is designed for HNDL-resistant key establishment through NIST-standardised ML-KEM.**

It should not be called "HNDL-proof", and the novel Entropic constructions should not be described as independently proven post-quantum primitives.

The defining statement

The shortest accurate definition is:

> **Entropic Fusion = Entropic Notation + ML-KEM-768 + ML-KEM-1024, mutually bound in one legitimate root-recovery path.**

The next sentence is equally important:

> **AES-256-GCM is the conventional authenticated data-encryption layer used by EFI after key establishment. It is not what the word Fusion refers to.**

Why fusion, not layers

A simple layered construction can often be described as "break layer A, then move on to layer B". That was not the design target for Entropic Fusion. The purpose of the binding work was to prevent Entropic Notation from becoming a decorative outer wrapper around an otherwise independent KEM system.

In the current frozen root:

1. the Entropic witness must verify;
2. successful verification derives the fusion secret;

3. that fusion secret releases both wrapped root KEM private keys;
4. ML-KEM-768 and ML-KEM-1024 are decapsulated; and
5. both KEM shared secrets, the EF-gated contribution, the transcript and application context are combined into the same root key.

This is what the project means by **fusion**. The components participate in one dependency graph on the official recovery path.

The research caveat is also important. If an external oracle simply hands an attacker both raw B.23.1 KEM shared secrets, the current EF-gated contribution can be recomputed from those secrets plus public transcript/commitment material. So the implementation is fused, but the project has not yet proved that every conceivable attack requires an independently secret Entropic break in addition to both KEM breaks. That stronger property is the next root target.

The simple EFI stack

The complete product can be read in two lines:

> **Entropic Notation + ML-KEM-768 + ML-KEM-1024 = Entropic Fusion**
>
> **Entropic Fusion + Entropic Tetration/Pentation + Entropic Chain + AES-256-GCM + metadata-private streaming + optional deniable storage = EFI**

This distinction keeps the public explanation simple without losing the actual architecture.

2. What Entropic Notation means in the cryptographic root

Entropic Notation began as a way of representing large arrangement spaces in which local relations, orientation and global consistency matter together. The current cryptographic root is not the 1,024-piece physical Born Solved puzzle. It is a frozen 4D n=3 research construction developed specifically for the cryptographic branch.

The current B.23.1 Entropic relation has the following structural parameters.

Quantity	Current value	Meaning
Dimensions	4	Four coordinate axes
Grid length per axis	3	n = 3
Positions	3^4 = 81	Complete placement positions
Pieces	81	One piece must occupy each position
Faces per piece	8	Positive and negative face on each of four axes
Proper orientations per piece	192	2^(4-1) x 4!
Required true adjacencies	216	4 x (3-1) x 3^3
Equation channels per	2	Face and centre

Quantity	Current value	Meaning
adjacency		
Materialised equations	432	216 x 2
Modulus	q = 257	Modular arithmetic domain
Hidden algebra dimension	64	Secret vector length
Noise/tolerance	+/-2	Current relation tolerance

Each public piece carries visible face-pattern information and modular shares. It does **not** publish a conventional standalone matrix of ready-made algebraic samples. A proposed pair of opposing faces, together with its full adjacency transcript, is required before the implementation materialises the corresponding algebraic equation.

The transcript includes the proposed pieces, positions, orientations, local face indices, world direction and visible face patterns. That structural transcript deterministically produces an equation coefficient vector and mask. The two endpoint shares then produce the equation right-hand side.

The intended dependency is therefore circular in the useful sense:

1. A candidate geometry is needed to determine which algebraic equations exist.
2. The hidden algebraic state is needed to determine whether that candidate geometry is valid.
3. The complete private witness must satisfy the whole geometry and all materialised equations before the fusion secret is released.

This coupling is the experimental core of Entropic Notation as used in Entropic Fusion.

3. Numerical scale of the Entropic domains

The placement and orientation domain is

81! x 192^81

which is approximately

5.136 x 10^305 arrangements.

The base-2 logarithm of that raw counting domain is approximately **1,015.55 bits**.

The hidden algebraic domain is

257^64

which is approximately

1.721 x 10^154 states,

with a raw counting logarithm of approximately **512.36 bits**.

A naive Cartesian multiplication gives

(81! x 192^81) x 257^64 approximately 8.838 x 10^459 combinations,

or approximately **1,527.91 bits of raw counting space**.

These are structural counting figures only. They are not a measured attack cost, not a lower bound on cryptanalysis, and not a claim of 1,527.91-bit security. Structure, constraints, algebraic shortcuts, SAT/SMT encodings, symmetry, meet-in-the-middle techniques or other attacks may reduce effective work dramatically. The purpose of publishing the raw domains is to define the object being studied, not to turn the object into an unsupported security rating.

4. From Entropic Notation to a fusion secret

Identity generation constructs an Entropic public instance and its private witness. The private witness contains two essential components:

1. the complete piece placement and orientation map; and
2. the hidden 64-component modular secret.

On recovery, the verifier performs the following checks.

1. The witness must cover all 81 positions.
2. Every piece must appear exactly once.
3. Every orientation must be valid.
4. Each of the 216 true adjacencies is traversed.
5. For every true adjacency, a face equation is materialised from the adjacency transcript and checked.
6. For every true adjacency, an independently domain-separated centre equation is materialised and checked.
7. All 432 equations must fall within the configured tolerance.
8. The verified layout transcript, equation transcript, public-instance hash and hidden secret are committed into a SHAKE-256 derivation.
9. The result is a fresh **32-byte / 256-bit fusion secret** for that identity.

The public identity contains only a SHA-256 commitment to this fusion secret. The private witness is required to reproduce it through the official verification path.

5. The ML-KEM side of Entropic Fusion

The B.23.1 root creates two standardised post-quantum KEM keypairs during identity generation:

Root leg	Parameter set	Ciphertext size	Shared-secret size
Root KEM A	ML-KEM-768	1,088 bytes	32 bytes
Root KEM B	ML-KEM-1024	1,568 bytes	32 bytes

Both public KEM keys are included in the public identity.

Both private KEM keys are encrypted with AES-GCM under the Entropic fusion secret. This is the first direct coupling point between Entropic Notation and ML-KEM.

The legitimate receiver cannot simply read the KEM private keys from the private identity. The official path is:

1. reconstruct the Entropic instance;
2. verify the complete Entropic witness;
3. derive the 256-bit fusion secret;
4. verify its public commitment;
5. use the fusion secret to unwrap the ML-KEM-768 private key;
6. use the same fusion secret to unwrap the ML-KEM-1024 private key;
7. decapsulate the sender's two KEM ciphertexts.

This is why the architecture is more than "Entropic Notation sitting next to ML-KEM". The Entropic relation participates directly in releasing the private material required by the official KEM recovery path.

6. The fused root key

For each file operation, the sender encapsulates independently to both root public keys and obtains:

- a 32-byte ML-KEM-768 shared secret; and
- a 32-byte ML-KEM-1024 shared secret.

The B.23.1 implementation also computes an EF-gated session contribution:

EF_session = SHA-256(domain || fusion_commitment || transcript_hash || ML-KEM-1024_shared_secret)

The final B.23.1 root combiner then binds:

1. the ML-KEM-768 shared secret;
2. the ML-KEM-1024 shared secret;
3. the EF-gated session contribution;
4. the complete ciphertext transcript; and
5. the application context.

A domain-separated HMAC-SHA256 extract/expand construction produces the final **32-byte / 256-bit Entropic Fusion root key**.

The root key is the starting state for Entropic Tetration's finite virtual-recursion engine.

7. What the fusion does and does not currently prove

The current root has a genuine implementation-fusion property:

- on the official private path, the Entropic witness must verify before the wrapped root ML-KEM private keys can be recovered;
- the public Entropic commitment and complete ciphertext transcript are bound into the EF-gated session contribution; and
- the final root key depends on both KEM sessions and that bound contribution.

However, B.23.1 also has a recorded limitation.

The entropy of the current EF-gated session contribution comes from the ML-KEM-1024 shared secret. The Entropic commitment and transcript are public bindings. Therefore, if an external oracle directly supplies both raw B.23.1 ML-KEM shared secrets, the current EF-gated contribution can be recomputed without possession of an independently secret third Entropic value.

Accordingly, the correct current claim is:

> **B.23.1 implements a fused Entropic-witness-gated ML-KEM recovery path. It does not yet establish that the Entropic layer contributes an independently quantified third hardness source once both raw KEM shared secrets are externally supplied.**

This limitation is the principal research target for the next root-generation branch.

8. EFI is the system built on Entropic Fusion

Entropic Fusion Integrated, or EFI, takes the 256-bit fused root key and builds a complete file-encryption and encrypted-storage system around it.

A normal EFI operation is one user action that runs the required internal stack automatically. The application does not require the user to execute each cryptographic layer manually.

The current high-level flow is:

Recipient public identity
-> **Entropic Fusion root: Entropic Notation + ML-KEM-768 + ML-KEM-1024**
-> **Standard / Tetration / Pentation finite state evolution**
-> **independent guard ML-KEM-1024 + Entropic Chain**
-> **context-bound inner EFI key**
-> **AES-256-GCM file/control protection**
-> **EFIPRIV3 metadata-private streaming**
-> optional **EFIPRIV4 deniable dual profile + duress credential**

The home-screen statement "One application. Whole EFI stack." refers to this complete automatic operation.

9. Numbered EFI encryption path

The following sequence defines the normal metadata-private standalone file path.

1. Read the recipient public identity

EFI loads the recipient's public Entropic Fusion/ET identity and the independent guard ML-KEM-1024 public key. The public identity fingerprint is used internally as a binding value but is hidden inside the final metadata-private container.

2. Generate the stream data key

EFI generates a fresh 32-byte / 256-bit stream data key for the actual user file. It also creates a 16-byte stream identifier and a 4-byte random nonce prefix.

3. Choose the privacy size class

The real source length plus the fixed private-control allowance is mapped to a graduated protected size class. The current policy uses an 8 MiB minimum and progressively coarser classes for larger files.

4. Stream the file

The real file is processed in **2,097,152-byte / 2 MiB** plaintext records. AES-256-GCM encrypts and authenticates each record with a nonce formed from the random 4-byte prefix plus a 64-bit record index.

5. Encrypt the privacy padding

After the real file ends, EFI writes zero plaintext to the rest of the selected size class. Those zeros are encrypted and authenticated exactly like real data. AES-GCM makes the padding opaque on disk, so random plaintext filler would provide no additional confidentiality benefit.

6. Build the secret stream-control record

Protected control information includes the original filename, exact real byte count, source SHA-256, stream data key, nonce prefix, stream identifier, chunk size and selected protected size class.

7. Establish the independent guard leg

A separate **ML-KEM-1024** encapsulation produces another 32-byte shared secret. HKDF-SHA256 binds it to its KEM ciphertext and recipient context to derive the 32-byte chain secret.

This guard KEM is separate from the two root KEMs described above.

8. Generate the root-binding secret

EFI independently generates a random 32-byte root-binding secret. This secret must later be recovered through the Entropic Fusion/ET path.

9. Establish one complete Entropic Fusion root

The B.23.1 root performs the Entropic Notation + ML-KEM coupling defined in Sections 2 through 7 and produces the 32-byte root key.

10. Apply the Entropic Engine switch

The user chooses one of three finite state-evolution modes.

Mode	Example	Behaviour
Standard	user-selected steps	direct finite sequential transitions
Tetration	2^^4	65,536 finite virtual transitions
Pentation	2^^^3	65,536 finite virtual transitions

The transition state remains 32 bytes. The transition count is a work/sequencing parameter, not a security-bit count.

The final virtual key protects the independent root-binding secret with AES-256-GCM.

11. Traverse Entropic Chain

The guard-derived chain secret drives the Entropic Chain. One cycle contains five mathematical families:

1. subset-sum;
2. XOR relation;
3. modular-sum relation;
4. ternary vector relation; and
5. permutation assignment.

The default profile uses 20 cycles, giving **100 sequential stages**, hence EC100. The current maximum profile is 256 cycles or **1,280 stages**.

12. Derive the inner EFI final key

HKDF-SHA256 combines the chain secret, chain terminal token, independently recovered root-binding secret and a commitment to the complete inner protocol context. The result is a 32-byte inner EFI final key.

13. Protect the stream-control record

AES-256-GCM encrypts the secret stream-control record under the inner EFI final key.

14. Build the fixed metadata-privacy buffer

The complete readable inner EFI capsule is compressed, framed with its true and compressed lengths, then zero-padded to a fixed **4,194,304-byte / 4 MiB** control plaintext.

The current maximum 256-cycle profile is regression-tested to fit this region with more than 1 MiB of required spare capacity.

15. Apply the outer privacy KEM

EFI performs a fresh outer **ML-KEM-1024** encapsulation to the recipient's guard public key. HKDF-SHA256 converts that shared secret into a fresh 32-byte outer control key.

This is a metadata-privacy wrapper. It is not counted as an additional independent Entropic hardness source.

16. Encrypt the fixed control region

AES-256-GCM encrypts the full 4 MiB control region with a fresh 12-byte nonce. The resulting encrypted control record is 4 MiB plus a 16-byte authentication tag.

17. Write the EFIPRIV3 framing

The public framing identifies EFIPRIV3, its version and the fixed outer KEM ciphertext length. The outer ML-KEM-1024 ciphertext and control nonce are public. The descriptive inner metadata is not.

18. Use an opaque output filename

The GUI suggests a random filename of the form EFI-<16 hexadecimal characters>.efi. The original source filename is restored only after successful private recovery.

10. Key hierarchy: every important key and what it does

EFI deliberately uses different secrets for different responsibilities.

Key or secret	Size	Origin	Purpose
Entropic fusion secret	32 bytes	verified Entropic witness	wraps both root KEM private keys
Root ML-KEM-768 shared secret	32 bytes	ML-KEM-768	root-combiner input
Root ML-KEM-1024 shared secret	32 bytes	ML-KEM-1024	root-combiner and EF-gated contribution input
EF-gated session contribution	32 bytes	ML-KEM-1024 secret + EF commitment + transcript	binds Entropic context into B.23.1 root
Entropic Fusion root key	32 bytes	B.23.1 combiner	initial ET virtual state
Virtual final key	32 bytes	Standard/ Tetration/ Pentation	protects root-binding secret

Key or secret	Size	Origin	Purpose
Root-binding secret	32 bytes	independent random generation	independent inner-final-key requirement
Guard ML-KEM shared secret	32 bytes	independent ML-KEM-1024	input to chain-secret KDF
Chain secret	32 bytes	HKDF from guard KEM	drives Entropic Chain
Chain terminal token	32 bytes	sequential chain traversal	inner-final-key requirement
Inner EFI final key	32 bytes	HKDF of chain/root/context inputs	protects stream-control record
Stream data key	32 bytes	fresh random generation	AES-256-GCM file and padding records
Outer privacy KEM shared secret	32 bytes	fresh ML-KEM-1024	input to outer-control KDF
Outer control key	32 bytes	HKDF	encrypts fixed 4 MiB control region

The repeated 32-byte size does not mean these values are interchangeable. They are generated or derived under different domains and have different lifetimes and purposes.

11. Privacy buffers

EFI uses two independent privacy-buffer concepts in EFIPRIV3.

Privacy buffer A: fixed 4 MiB encrypted control region

The complete inner EFI protocol capsule contains sensitive descriptive information needed for legitimate recovery and validation, including the original filename, exact source length, recipient fingerprint, selected Entropic mode, hyperoperation notation, transition count, Entropic Chain profile and internal KEM/root records.

Publishing that JSON directly would leak useful metadata even if the file payload remained encrypted.

EFI therefore:

1. compresses the complete inner capsule;
2. frames the true and compressed lengths inside the protected plaintext;
3. fills the remainder of a fixed 4 MiB control region with zeros; and

4. encrypts and authenticates the entire region under a fresh outer ML-KEM-derived AES-256-GCM key.

The public container therefore exposes a constant control-region size rather than the size or content of the internal protocol description.

Privacy buffer B: graduated encrypted file padding

Exact file length can itself reveal useful information. EFI therefore pads the real source to a coarse size class.

Current classes are:

Protected total before small framing overhead	Policy
<= 8 MiB	8 MiB minimum
> 8 MiB to 64 MiB	next 8 MiB
> 64 MiB to 512 MiB	next 32 MiB
> 512 MiB to 4 GiB	next 128 MiB
> 4 GiB	next 256 MiB

Representative validated size-policy examples are:

Real source	Protected class	Approximate visible EFI size
2.14 MiB	8 MiB	about 8.002 MiB
20 MiB	24 MiB	about 24.002 MiB
100 MiB	128 MiB	about 128.003 MiB
500 MiB	512 MiB	about 512.005 MiB
2 GiB	2.125 GiB	about 2.125 GiB

The padding is encrypted zero plaintext. It is not intended to add cryptographic key strength. It hides exact length within a public size class.

12. EFIPRIV3 public byte layout

The current standalone metadata-private container has the following public structure.

Field	Length	Visibility
Magic	8 bytes	public: EFIPRIV3
Version	1 byte	public
Flags	1 byte	public
KEM ciphertext length	2 bytes	public
Outer ML-KEM-1024 ciphertext	1,568 bytes	public, fresh per file

Field	Length	Visibility
Control nonce	12 bytes	public, fresh per file
Encrypted fixed control	4,194,320 bytes	opaque
Data records	2,097,168 bytes each	opaque

A knowledgeable observer can identify the EFI format, infer the coarse size class and fingerprint the 1,568-byte outer KEM field as consistent with ML-KEM-1024. The observer cannot read the original filename, exact source byte count, recipient fingerprint, Entropic mode or chain profile from a correctly produced EFIPRIV3 container.

EFIPRIV3 is a file-container privacy design, not a traffic-analysis anonymity system. Filesystem timestamps, storage-provider logs, endpoint compromise, process telemetry and copies outside the container are beyond the format's protection boundary.

13. Optional EFIPRIV4 deniable dual-profile container

EFI 1.0.0 beta adds an optional deniable mode. Normal encryption remains EFIPRIV3. Deniable mode is explicitly selected and produces EFIPRIV4.

Every EFIPRIV4 container contains **two equal opaque slots**.

- Slot 0 contains the ordinary or decoy EFI profile.
- Slot 1 always exists. It contains either random cover or a separately protected hidden EFI profile.

The public header does not contain a "hidden profile present" flag. The user selects the reserve geometry independently. Hidden content is not allowed to silently enlarge the public container, because doing so would leak information about hidden-profile capacity.

The hidden profile passphrase is strengthened with Argon2id using 64 MiB memory, time cost 3 and parallelism 1, then combined with a fresh ML-KEM-derived slot secret through HKDF. Slot contents are encrypted with AES-CTR and authenticated with a separate HMAC key derived for that slot.

The deniable format is an optional storage feature. It does not alter the definition of Entropic Fusion itself.

14. Deliberate duress / hidden-profile self-destruct credential

EFIPRIV4 can optionally configure a deliberate duress credential.

This is not an ordinary wrong password. A typo or incorrect credential is non-destructive.

The credential paths are:

1. ordinary decoy path: recover the decoy profile;
2. hidden credential: recover the hidden profile;
3. duress credential: recover the decoy profile successfully and replace the current hidden-slot recovery material; and

4. any unrelated wrong credential: fail without destroying data.

When the duress credential is recognised, EFI replaces the live hidden-slot ML-KEM ciphertext, salt, IV and authentication material with fresh random bytes. The large hidden ciphertext body is not rewritten. The result is fast cryptographic erasure of recovery capability for the **current container copy**.

The deliberately narrow claim is:

> **The configured duress credential opens the decoy profile and destroys live recovery of the hidden profile from that current EFIPRIV4 container copy.**

It does not erase backups, snapshots, cloud version history, filesystem journals, SSD remanence or previously copied containers. It does not destroy the decoy profile and it does not wipe the complete .efi file.

15. Conventional and post-quantum components: correct language

EFI deliberately contains both established conventional cryptography and standardised post-quantum key establishment because they solve different parts of the same long-term confidentiality problem. The conventional cipher protects the data efficiently; the post-quantum KEM protects the establishment and transport of the secrets needed to use that cipher. This division is central to the HNDL threat model.

Standardised / established components used by EFI

- ML-KEM-768 and ML-KEM-1024 for post-quantum key establishment;
- AES-256-GCM for authenticated encryption;
- HKDF-SHA256 for context-bound derivation;
- HMAC-SHA256 for selected sequential/state and authentication roles;
- scrypt for private-identity password protection; and
- Argon2id for selected chain and deniable-profile password gates.

Experimental project constructions

- Entropic Notation / the Entropic relation;
- Entropic Fusion as the coupled root architecture;
- finite virtual Tetration/Pentation state evolution; and
- Entropic Chain / EC100.

It is defensible to describe EFI as providing **post-quantum key establishment through NIST-standardised ML-KEM, with conventional AES-256-GCM authenticated data encryption**.

It is not defensible at this stage to say that the novel Entropic constructions themselves have been formally proven post-quantum secure.

16. Security interpretation

The correct security posture for the current project is deliberately conservative.

What is established by the implementation and regression work

- The Entropic witness is structurally required by the official B.23.1 private recovery path before the wrapped root KEM private keys are released.
- The B.23.1 root key changes if any of its bound inputs change.
- EFI performs exact authenticated round trips in the validated Windows build.
- EFIPRIV3 hides the tested descriptive metadata from public container bytes.
- The same payload can be emitted under Standard, Tetration and Pentation with the same public size while producing distinct ciphertext bodies.
- Large files are processed in bounded 2 MiB records rather than loaded into memory as one object.
- EFIPRIV4 has fixed dual-slot geometry, a non-destructive wrong-credential path and a deliberate duress path that returns the decoy while destroying current hidden-profile recovery.

What remains a research question

- the effective attack complexity of the Entropic relation;
- whether there are structural or algebraic shortcuts far below its raw counting domains;
- whether the Entropic side can be redesigned to contribute an independently necessary secret even when both raw KEM shared secrets are exposed;
- the best formal security model for the coupled root; and
- the extent to which Tetration/Pentation and Entropic Chain contribute useful defensive properties rather than merely sequential work and diversity.

17. The next cryptographic research target

The application feature set can now stabilise independently from the research root.

The next root target is conceptually simple:

> **Even if an attacker is handed both raw root ML-KEM shared secrets, reconstruction of the final Entropic Fusion root key should still require an independently secret value obtainable only through the Entropic witness/relation.**

That would convert the current implementation-fusion property into a stronger form of cryptographic dependency.

A future research branch should therefore introduce an Entropic-derived secret contribution that is not a deterministic function of the ML-KEM shared secrets plus public transcript material, then attack that branch under explicit negative controls.

18. Reference implementation lineage

The current defining implementation is composed from frozen research branches:

- Entropic Fusion Beta 1.1.1 / B.23.1 root;
- Entropic Tetration ET 0.13;

- Entropic Chain core;
- EFI 0.5 inner integrated protocol;
- EFIPRIV3 metadata-private streaming; and
- EFIPRIV4 deniable dual-profile wrapper.

The cryptographic modules carried into EFI 1.0.0-beta.5 are unchanged from the Windows-validated EFI 0.10.1 baseline. The beta.5 release candidate locks the HNDL motivation, dual-ML-KEM fusion definition and non-layered dependency language across the application, Help Centre and papers without changing the validated cryptographic modules.

Validated EFI 0.10.1 Windows artifacts include:

Artifact	SHA-256
EntropicFusionIntegrated.exe	4b8eacd739f1dfe4e7085ae50f19c350ca7faa208074935fbda0205329d1fbb7
EntropicFusionHost.exe	1606247bc46b9a55539a1416b0cef196cf1679fc100cda1cb052eec9be4d92fe
Windows installer	aebe19f50abac351e6eb1ce3bc4b4ddfd4e3687ec8cd2a5ac2cde7b2d8b8fbfb
Portable ZIP	ab882dcf6cc9085112c08231fe8dd503e92aabf59c14f579f5f4ede876be2251
Bundled WinFsp MSI	073a70e00f77423e34bed98b86e600def93393ba5822204fac57a29324db9f7a

19. Final definition

Entropic Fusion should therefore be described in one precise paragraph:

> **Entropic Fusion is an experimental coupled cryptographic root developed for long-term confidentiality in the Harvest Now, Decrypt Later era.** It binds the project's Entropic Notation relation with two NIST-standardised post-quantum key-establishment sessions, ML-KEM-768 and ML-KEM-1024, inside the same legitimate private recovery dependency. The verified Entropic witness derives the fusion secret that releases both wrapped root KEM private keys, while the root combiner binds both resulting KEM shared secrets, the EF-gated contribution, the ciphertext transcript and application context into one 256-bit root key. EFI builds on that fused root with Entropic Tetration/Pentation, Entropic Chain, conventional AES-256-GCM authenticated data encryption, metadata-private streaming, privacy padding, deniable profiles and optional duress key erasure. ML-KEM supplies the standardised post-quantum HNDL-resistant key-establishment component; the novel Entropic

constructions remain experimental research objects pending independent cryptanalysis and formal security analysis.

That is the defining architecture.

Appendix A. Numerical reference

Parameter	Current value
Entropic positions	81
Pieces	81
Faces per piece	8
Proper orientations per piece	192
Required true adjacencies	216
Materialised equations	432
Modulus	257
Hidden algebra dimension	64
Raw geometry domain	81! x 192^81 approximately 5.136 x 10^305
Raw algebra domain	257^64 approximately 1.721 x 10^154
Naive joint counting domain	approximately 8.838 x 10^459, scale only
Root KEM A	ML-KEM-768
Root KEM B	ML-KEM-1024
Independent guard KEM	ML-KEM-1024
Outer privacy KEM	ML-KEM-1024
KEM shared-secret size used	32 bytes
Fusion/root/chain/data/control key sizes	32 bytes
AES-GCM nonce	12 bytes
AES-GCM tag	16 bytes
Tetration demonstration	2^^4 = 65,536 transitions
Pentation demonstration	2^^^3 = 65,536 transitions
ET executable cap	10,000,000 transitions
Default Entropic Chain	20 cycles = 100 stages
Maximum current Entropic Chain	256 cycles = 1,280 stages
Stream plaintext record	2 MiB
Fixed EFIPRIV3 control plaintext	4 MiB
EFIPRIV4 slot count	2 equal opaque slots
Hidden/duress passphrase minimum	12 characters
EFIPRIV4 Argon2id memory	64 MiB

Parameter	Current value
EFIPRIV4 Argon2id time cost	3

Appendix B. Terminology

HNDL (Harvest Now, Decrypt Later): an attack strategy in which encrypted material is collected today and retained for future decryption after advances in quantum computing or cryptanalysis. EFI addresses this threat through post-quantum ML-KEM key establishment.

Entropic Notation: the project's structural representation of coupled arrangement/orientation and materialised relation constraints. In the current cryptographic root it is instantiated by the 4D n=3 geometry/algebra relation.

Entropic relation: the executable verifier that tests the complete Entropic witness, materialises face and centre equations from candidate adjacencies and derives the fusion secret after successful verification.

Entropic Fusion: the coupled root architecture that binds the Entropic relation to ML-KEM key establishment on the legitimate recovery path.

EFI: Entropic Fusion Integrated, the complete application architecture built on Entropic Fusion.

ET: Entropic Tetration, the finite Standard/Tetration/Pentation state-evolution engine.

EC / EC100: Entropic Chain. EC100 is the normal 20-cycle, 100-stage profile.

Fusion secret: the 256-bit secret derived after successful Entropic witness verification and used to wrap/release the two root ML-KEM private keys.

Root key: the 256-bit output of the B.23.1 root combiner and input to the ET virtual state engine.

Guard KEM: the independent ML-KEM-1024 leg used to derive the Entropic Chain secret and also reused as the recipient key for the fresh outer EFIPRIV3 privacy encapsulation.

Privacy buffer: either the fixed encrypted 4 MiB EFIPRIV3 control region or the graduated encrypted padding used to hide exact source length within a size class.

Deniable profile: the optional EFIPRIV4 second profile hidden inside a fixed opaque slot that also exists as random cover when unused.

Duress credential: an optional deliberate credential that returns the decoy profile and destroys live hidden-profile recovery material in the current EFIPRIV4 container copy.

Research status: This paper defines the current implementation architecture. It is not a formal cryptographic proof and does not assign a formal security-bit rating to the novel Entropic constructions.